

Comparison and prediction of startup success criteria using various statistical and machine learning models

Anchal Sondhi (627, asondhi7@gatech.edu)

Bhujanga K Adapa (836, badapa6@gatech.edu)

Sumeyye Adanur (880, sadanur3@gatech.edu)

December 23, 2025

Abstract

Startups play a pivotal role in driving innovation and economic growth, yet a large amount of them fail. Accurately predicting the likelihood of startup success can help investors in allocating resources effectively and make important investment decisions. This project investigates the prediction of startup success using the Kaggle Startup Success Prediction dataset, which includes 923 observations and 49 features spanning financial, organizational, and geographic factors. Exploratory data analysis highlighted key patterns, that includes the importance of milestone achievement, team size, and professional networks, as well as geographic and funding-related trends. Various machine learning and statistical models - logistic regression, decision tree, random forest, k-nearest neighbors, naive bayes, support vector machine, and XGBoost were trained on a 70-30 train-test split, with hyperparameter tuning via cross-validation. Model performance was evaluated using ROC-AUC, F1 score, and accuracy metrics, while class imbalance and SMOTE was tested in handful of models but not adopted for the whole analysis. Random forest emerged as the best-performing model, followed by decision tree and boosting, both achieving metrics better than the standard benchmarks ($\text{ROC-AUC} \geq 0.75$, $\text{F1} \geq 0.70$, $\text{accuracy} \geq 0.70$) in literature. Logistic regression, knn, naïve bayes, SVM, and XGBoost showed comparatively lower performance, highlighting the advantages of tree-based methods for capturing complex interactions in startup data. We also came up with a hybrid model approach that uses gradient boosting and logistic regression as the feedthrough to the random forest model. The performance of this model was comparable to the highest performing ones, but random forest was still the best performing out of all the models that we tested. Feature importance analysis consistently identified milestone timing and professional relationships as the most important predictors of startup success. These findings demonstrate the feasibility of predictive modeling for startup outcomes and provide important insights for investors to allocate resources more effectively.

Introduction

Startups represent driving force behind innovation including technological change that hold the promise of disruption throughout the entire business ecosystem, spanning small, medium and large enterprises. These startups also mark key milestones in a country's societal and economic developments. Unfortunately, roughly 8 to 9 out of 10 startups fail [1,2].

These ventures attract significant investment both by institutional and individual investors, directly and indirectly. In first and second quarters of 2025 alone, globally, startups reached approximately \$200 billion USD investments which is a significant amount of money and is growing rapidly across the globe [1,2].

The above reasons highlight the need to find or develop methods and models that can predict the likelihood of success of a startup based on specific parameters. Such findings could help investors make informed decisions on how to allocate capital for their investments and when to exit. This project explores various machine learning and statistical models and applies them to the Startup Success Prediction dataset from Kaggle with the goal of forecasting whether some startups will succeed or fail.

Problem Statement and Data sources

The problem is to develop a predictive framework that uses financial, organizational, and geographic features to forecast startup success, defined as an IPO or acquisition. The solution would address challenges like class imbalance and mixed-type data. Additionally, the model would identify key factors influencing success while providing insights for investment strategy and resource allocation.

The dataset is the [Startup Success Prediction](#) dataset sourced from Kaggle and is openly available to download. It contains 923 observations and 49 variables all for the states in the USA. Table 1 shows the broad categories of factors in the dataset. More detailed description of factors is available in table 8 in the appendix.

Table 1: Categories of factors and their data type

Factor category	Data Type
Category/Flag	int, object
Company id	object
Date/Time	object
Elapsed/Metrics	float
Finance	int
Geography	object, float, int
Relations	int
Status/Target	int, object

Proposed Methodology

EDA was carried out to understand class imbalances, feature distribution and correlation amongst various features. The training and prediction were done using Logistic Regression, Decision Tree, Random Forest, k-nearest neighbors, Naïve Bayes, Support Vector Machine, XGBoosting and a hybrid model. The training and testing was obtained from a 70-30 train-test split. Models were tuned using cross-validation and performance was evaluated using ROC-curve, AUC, F1 score and accuracy metrics.

The feature space was reduced from 49 to 20 features. These features were streamlined or removed considering that they don’t contain any information for models or they have redundant information that already is in other columns. For example, is_500 is a column that model can easily assume that if they are in this column then they are successful. This column can lead to prediction with inflated results.

Classification thresholds were compared against what literature proposes. Studies in venture capital and startup analysis suggest benchmarks for a “good” model: ROC-AUC ≥ 0.75 , F1 Score ≥ 0.70 accuracy ≥ 0.70 would indicate strong discrimination between successful and failed startups [4,5,6]. This framework not only forecasts startup outcomes but also identifies key factors influencing success.

Based on learnings from these standard models and their performance on metrics, a hybrid model is proposed that incorporates different kinds of these models.

In our analysis we considered an acquired startup as success and labeling it as 1 in the analysis while any other scenario as failed and labeled it as 0.

Analysis and Results

Class imbalance

We analyzed a dataset of 923 startup companies, of which 597 were successfully acquired and 326 failed. This yields a success-to-failure ratio of roughly 2:1, indicating a class imbalance that could influence predictive models to favor the majority class. See fig.1 for more information.

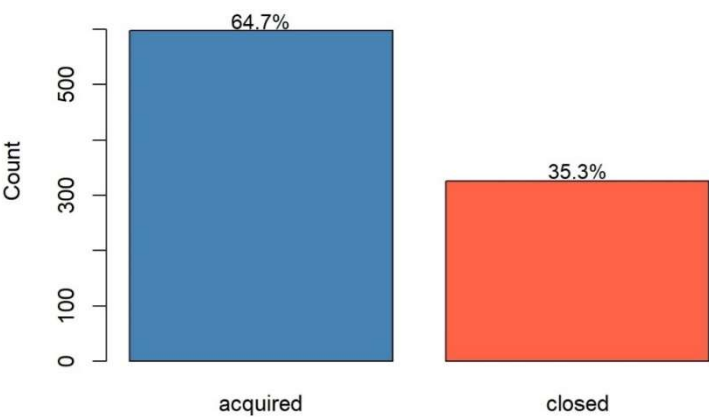


Figure 1: Bar chart showing class imbalance of acquired vs closed startups

To mitigate class imbalance, several techniques can be used without compromising model performance. Class weighting is one such technique where higher penalties are assigned to minority class errors. Resampling methods like SMOTE can also be used to generate synthetic minority samples that alleviates class imbalance issues.

One of the reviewers of our midterm report pointed out that this level of imbalance is usually manageable without aggressive resampling. We still tested SMOTE on a handful of models to be thorough. We compared the same models trained with and without SMOTE on the untouched test set. The results are shown in Table 2. We did not see a substantial accuracy difference and hence did not follow class imbalance practices for whole gamut of models.

Table 2: Table comparing the with SMOTE and without SMOTE comparison of Decision Tree and XGBoost models

Scenario	Model	Accuracy
Without SMOTE	Decision Tree	0.80
Without SMOTE	XGBoost	0.81
With SMOTE	Decision Tree	0.79
With SMOTE	XGBoost	0.80

Exploratory Data Analysis

The dataset contains various variables that can be broadly classified into finance, time elapsed, geography and relationships related categories. Following paragraphs detail some of the important ones from these categories.

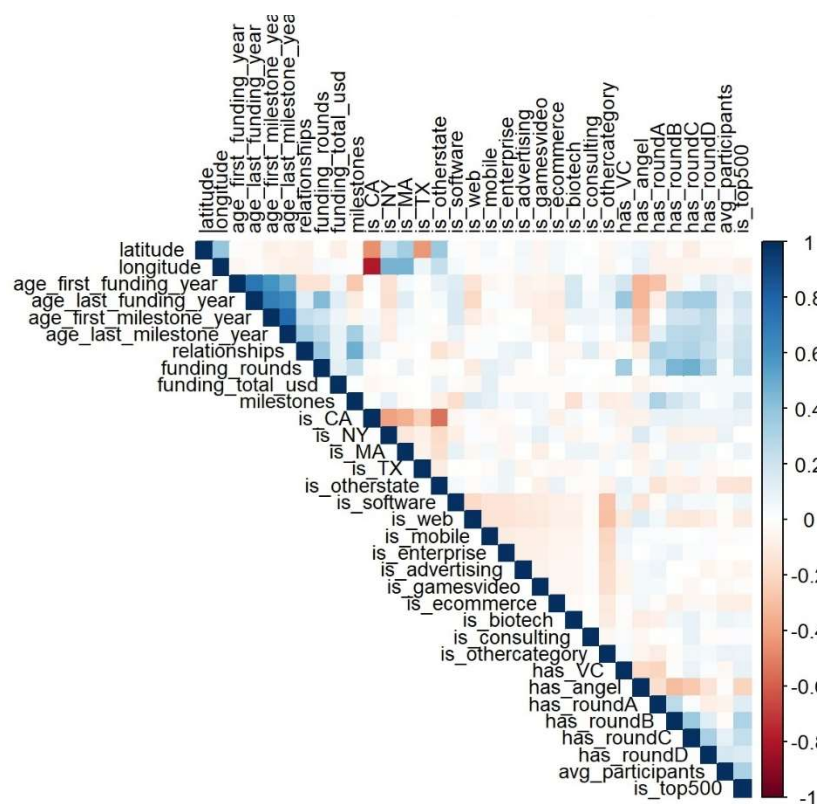


Figure 2: Correlation heatmap for factors

Figure 2 shows the correlation matrix amongst various parameters. A moderate positive correlation (0.51) exists between number of relationships and milestones achieved, confirming that larger, better-connected teams consistently reach more developmental goals.

Table 3 shows funding round and success rate. A clear pattern emerges from funding progression: success rate rises steadily with each subsequent funding round, from 73% for Round A to 85% for Round D. This shows that having more funding rounds has strong correlation to acquisition/success of a startup.

Table 3: Table for founding rounds vs success rates of startups

Funding Round	Success Rate
Round A	73%
Round B	77%
Round C	79%
Round D	85%

Bigger team size and deeper professional network also showed strong correlation to the success of a startup. Acquired startups had an average of 9.64 relationships, more than twice the average of 4.17 observed in closed companies.

Timing and achievement of milestones were found to be more important than funding amount in prediction of success. Acquired startups that reached their first milestone faster had survived significantly longer after their final recorded milestone. While total funding correlates with success, milestone-related variables ultimately dominated the final models.

Geographically (fig. 3), California has over 70% of all startups and thus dominates both acquired and closed outcomes. However, when ranked by average funding per startup, California comes at 10th place, and Washington and Indiana received substantially higher average investments per company.

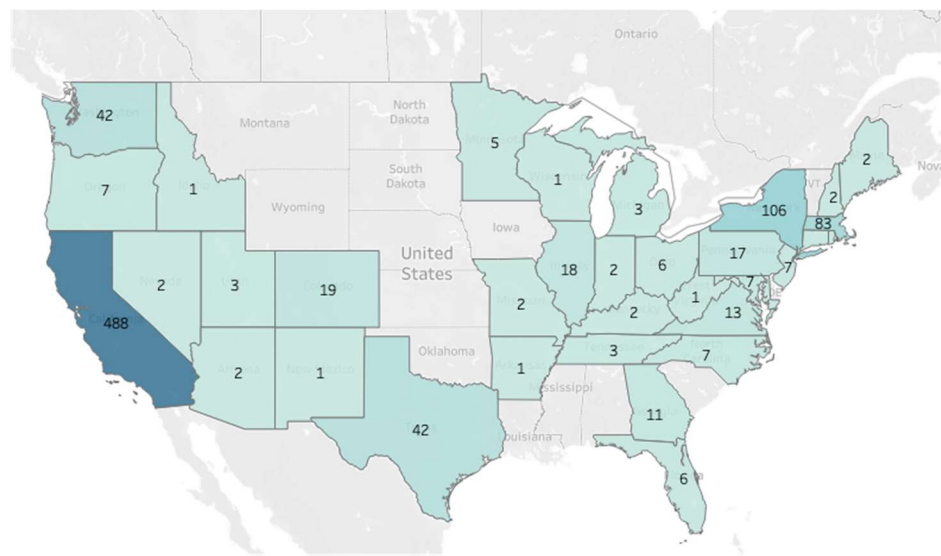


Figure 3: Geographic plot of how startups are distributed across USA

Startups without venture capital or angel investment slightly outperformed their funded counterparts (66.6–66.7% success rate versus 60.8% for VC-backed and 58.7% for angel-backed). This might point to the idea that more conservative growth strategies that exclude high burn rates and scaling are often encouraged by institutional investors.

Figure 4 shows the Q-Q plots of various numeric (total funding, funding rounds, first funding year and average participants) features and show right-skewed distributions and numerous outliers. This highlights the high variability of the startup ecosystem. Strong inter-correlations were seen for funding rounds, total funding, relationships, and milestones.

These exploratory findings aligned closely with the final modeling results and provided clear intuition for why milestone timing and team relationships are the most important predictors of acquisition success.

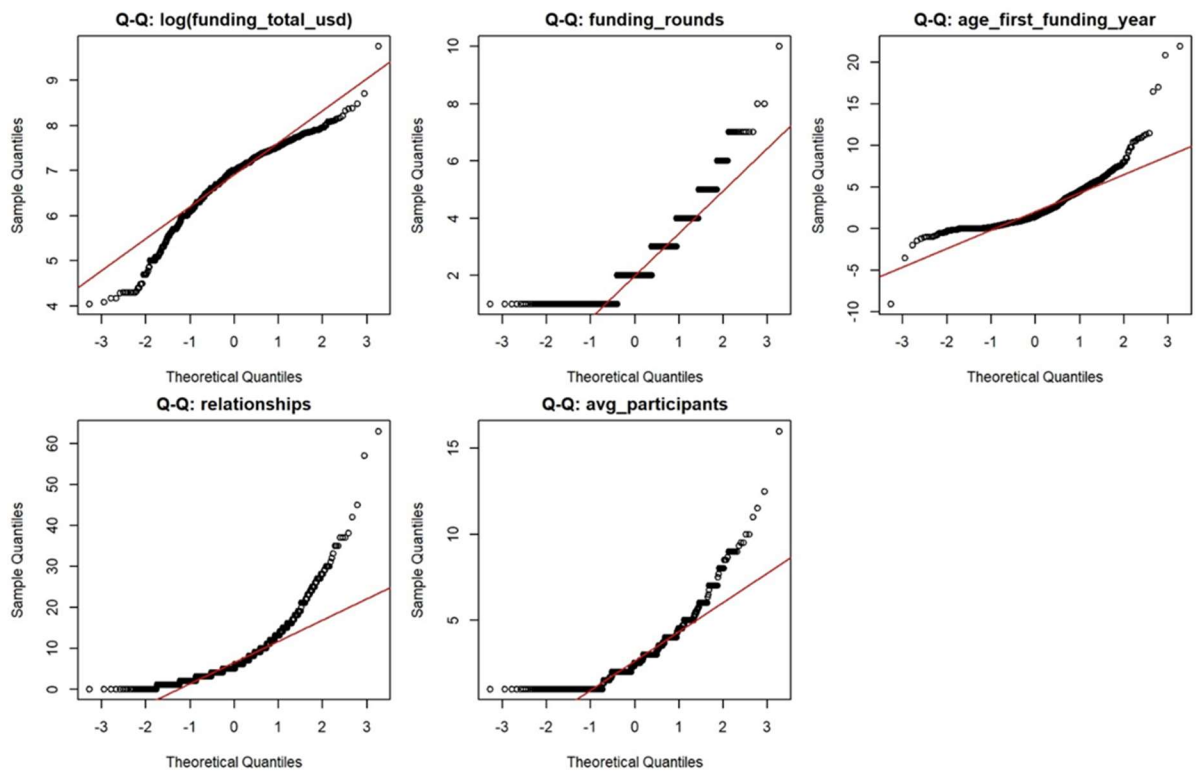


Figure 4: Q-Q plot for numeric data type factors

Model performance and comparison

Logistic Regression

Logistic Regression cannot handle categorical variables. Therefore, all categorical features were transformed into numerical form using one-hot encoding. All numerical features were standardized and then optimized using 5-fold cross-validation. GridSearchCV was used to optimize hyperparameters. Adding class_weight did not improve the evaluation results. The best model selected by GridSearchCV had:

- $C = 1$
- Penalty = L2 regularization
- Solver = lbfgs

Decision Tree

Decision Tree has good ability to handle mixed data types. Categorical features were one-hot encoded, numerical features standardized, and the tree was pruned using cost-complexity (cp). After 5-fold cross-validation, the optimal $cp = 0.008$ was chosen. The final model trained on the original data achieved the AUC of 0.84.

Random Forest

Random Forest was selected as a non-linear ensemble model capable of capturing complex interactions between features. Unlike Logistic Regression, Random Forest does not require strict assumptions such as linearity or scaled inputs. Adding class_weight did not improve the evaluation

results. RandomizedSearchCV was used to tune key hyperparameters with 5-fold cross-validation. See fig. 5

The best-performing configuration identified was as below:

- n_estimators = 200
- max_depth = 10
- min_samples_split = 10
- min_samples_leaf = 1
- max_features = 'sqrt'
- bootstrap = False

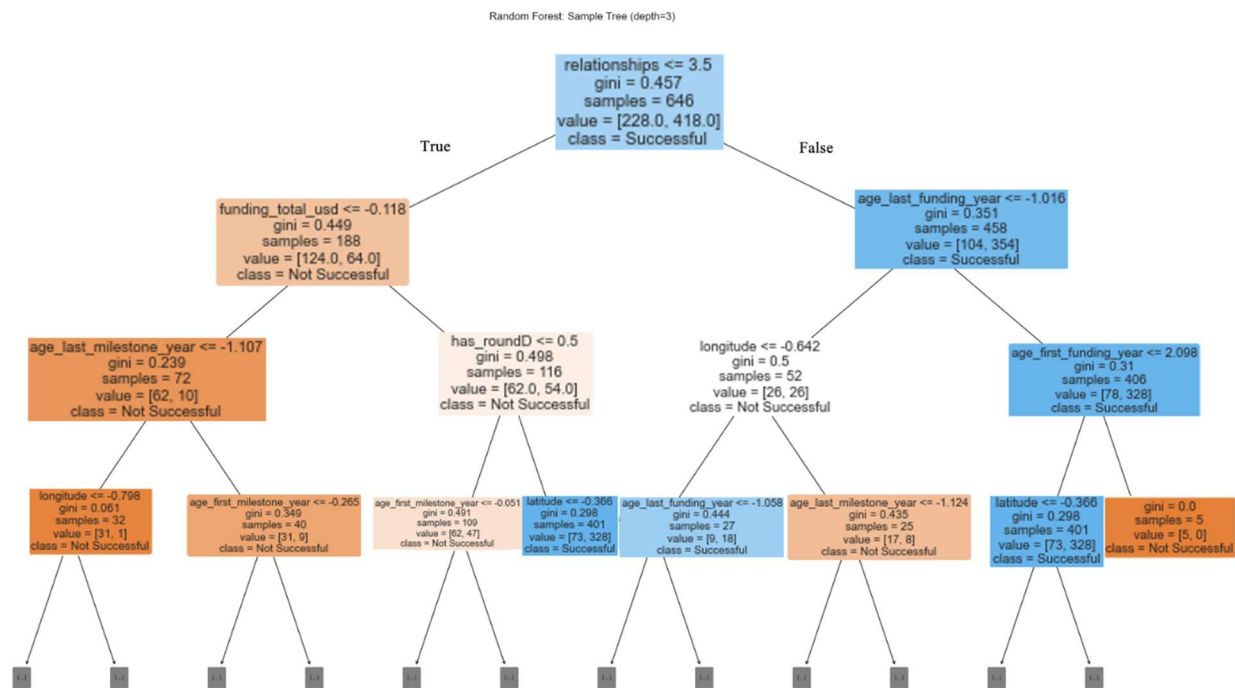


Figure 5: Random forest graph

K-Nearest neighbors

K-Nearest Neighbors (KNN) is a distance-based, non-parametric classifier. All numeric features were scaled, categorical features were one-hot encoded, and clusters were identified. The number of neighbors (K) was optimized via 5-fold cross-validation over values 1 to 20, with K=18 selected as the best. While KNN can capture local patterns effectively, it is sensitive to noisy features and treats all features equally, which limited its AUC compared to other models.

Naïve Bayes

Naive Bayes is a fast probabilistic model. Features were one-hot encoded and standardized, and the smoothing parameter was tuned via 5-fold cross-validation. Its AUC was not the best which also confirms that simple probabilistic models can suffer from class imbalance, while tree-based methods handle it better.

Support Vector machine

Support Vector Machine (SVM) was applied with scaled numeric features, one-hot encoded categorical features, and cluster-derived features. We used an RBF kernel (kernel='rbf'), probability estimates (probability=True), C=1.0, and gamma='scale'. Despite these settings, SVM underperformed compared to tree-based models due to sensitivity to feature scaling, noise, and class imbalance in the startup data.

XGboosting

XGBoost is based on gradient-boosting algorithm. Categorical features were one-hot encoded, and no feature scaling was required. Model was tuned using 5-fold CV and hyperparameters were fixed (learning rate = 0.05, max_depth = 6, subsample = colsample_bytree = 0.8).

Hybrid model

We tried two different approaches to come up with a hybrid model. One of them included taking an average of different kinds of model. The one that we feel worked the best was the second approach.

The second approach included hybrid stacking of the models by combining random forest, gradient boosting, and logistic regression. We selected a combination of tree based and probabilistic models because they were the best performing models in our analysis. The hybrid setup is such that the gradient boosting and logistic regression predictions, along with original features, are fed into random forest to learn optimal combinations. This approach uses each base learner's strengths: tree-based models capture nonlinear interactions, while logistic regression adds a linear perspective. Key parameters include 200 estimators for both the base random forest and the meta-random forest, class balancing for the trees, and passthrough enabled to feed the predictions from the two models. On the startup dataset, the hybrid model achieved good AUC, F1 score and precision values, but interestingly, a random forest performed slightly better.

Table 4 shows the AUC, F1 score and accuracy metrics obtained from these models. Random forest performed the best overall, followed by hybrid model and decision tree. We think the hybrid model was not the best performing model because the combination included pre-cursor models that were not the best performing ones in the original model-by-model comparison. This could have led to the degradation of hybrid-model's performance.

Fig. 6 shows the ROC curve for the whole gamut of models. The random forest, hybrid model and decision tree curves are stacked all the way to the left of the graph indicating good performance throughout the range of sensitivity and specificity values.

Table 4: Performance comparison for various models

Models	AUC	F1 score	Accuracy
Random forest	0.86	0.88	0.84
Hybrid-model	0.84	0.88	0.83
Decision tree	0.84	0.85	0.80
Boosting	0.83	0.87	0.81
SVM	0.8	0.83	0.77
Naive Bayes	0.8	0.79	0.65
Logistic regression	0.79	0.82	0.75
KNN	0.7	0.76	0.68

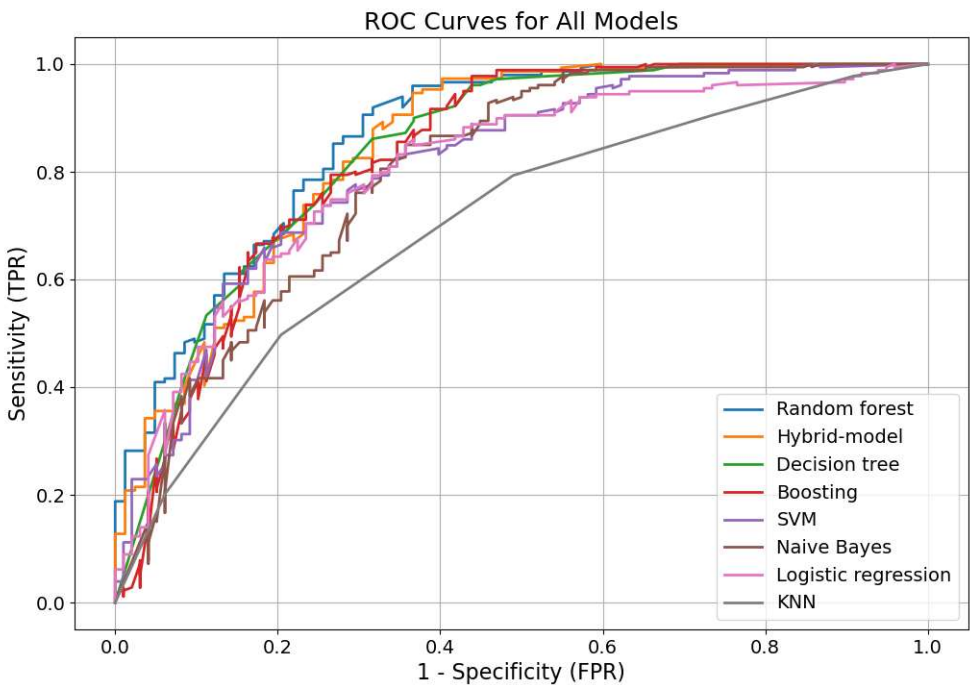


Figure 6: ROC curve for the models

Feature selection

Table 5 shows some of the features and their rankings with rank 1 being most important and subsequent rank numbers being less important. Based on this, most of the models regard “Age_last_milestone_year” as the most important feature out of all, while “Age_first_funding_year” was the least important.

Table 5: Feature selection outcome from some of the models. The features are ranked most important (rank 1) to less important (higher ranks)

Features	Logistic Regression	Random Forest	XGBoost	Decision Tree
Relationships	4	2	1	3
Age_last_milestone_year	1	1	2	1
Milestones	2	4	7	4
Funding_total_usd	5	5	5	5
Age_first_funding_year	3	6	6	6
Age_first_milestone_year	10	3	3	2

Conclusions

We were able to analyze Kaggle Startup Success Prediction dataset that had mixed data types to understand which features are key drivers of success for a startup and predict it using various statistical and machine learning models. The EDA analysis yielded important features that lead to higher success – higher funding amounts, multiple funding rounds, larger professional networks and multiple rounds of funding. Despite geographic distribution showing a high concentration of startups in California, the importance of factors like milestones and relationships show the importance of success irrespective of geographic location.

The analysis further showed a class imbalance (~65% acquired and ~35% closed) but a preliminary SMOTE analysis showed that such class imbalance did not have significant influence on the prediction performance. Thus we did not adopted class balancing methods for the analysis.

Model performance shows random forest model as the best performing model on the AUC, F1 score and accuracy metrics and is very closely followed by decision tree and hybrid models. Together these three models outperform all the other models. This shows the superior predictive performance of tree-based models compared to linear, distance-based, and probabilistic models. All the values of the metrics were well above the threshold values that we found in the literature - AUC-ROC >= 0.75, F-1 score >= 0.7, accuracy >=0.7 that work well for startup success prediction. This also implies that fundamentally most of the machine learning models can predict startup success.

We think the tree based models outperformed all other types of models because, one, our dataset contains lot of non-linear relationships and, second, there were several outliers. Tree-based models are good at analyzing complex relationships and are robust to outliers during splitting. Thus outperforming other types of models.

We also introduced a hybrid model that was a combination of random forest, gradient boosting, and logistic regression such that gradient boosting and logistic regression feed their predictions into the random forest. Based on the output from it, it came in at third after random forest and decision tree model. We believe this happened because the combination included a probability-based model (logistic regression) that impeded or degraded the overall model performance.

Feature importance analysis consistently identified milestone timing and professional relationships as the key predictors of success. This is also what we saw in the exploratory data insights. Overall,

the results suggest that predictive modeling using tree-based ensembles can reliably forecast startup outcomes and guide investment decisions.

Lessons Learned

This project highlighted the importance of hyperparameter tuning for model performance and showed that tree-based models particularly random forest, XGBoost, and the hybrid setup clearly outperformed linear, probabilistic, and distance-based approaches on the startup success prediction task. Even a basic combination ranked among the top performers, confirming that diverse models can provide excellent results.

We felt that the course content could be made more absorbable to where the slides are easier to interpret and videos are more than just reading the content of the slides.

Appendix

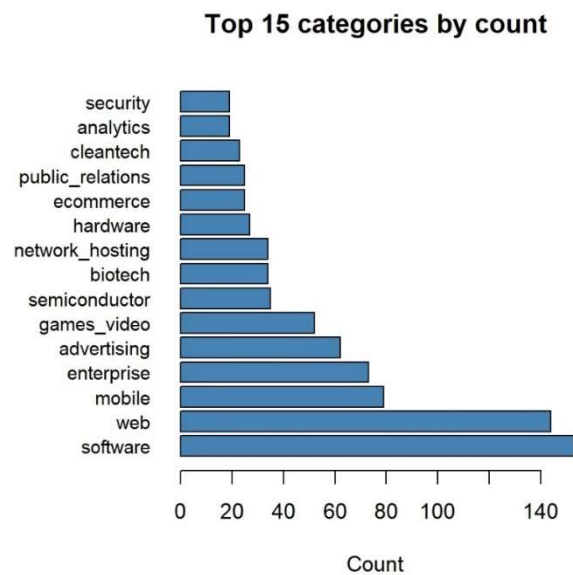


Figure 8: Categories of software companies

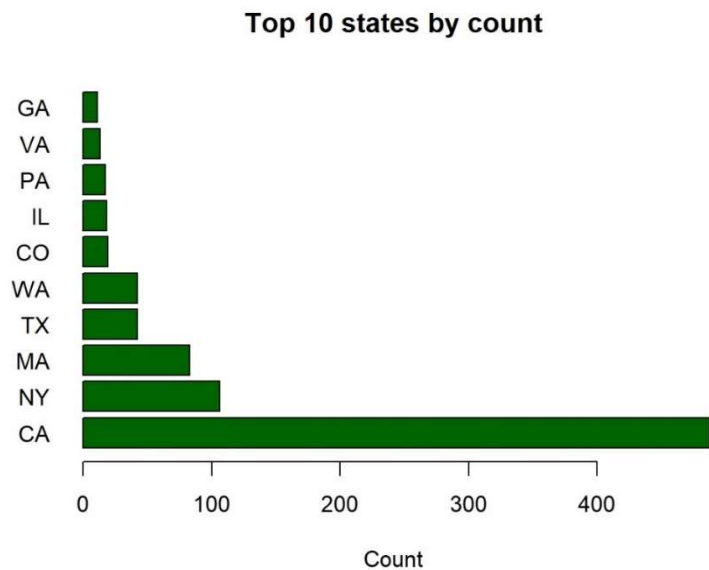


Figure 9: Top 10 states by location of startups

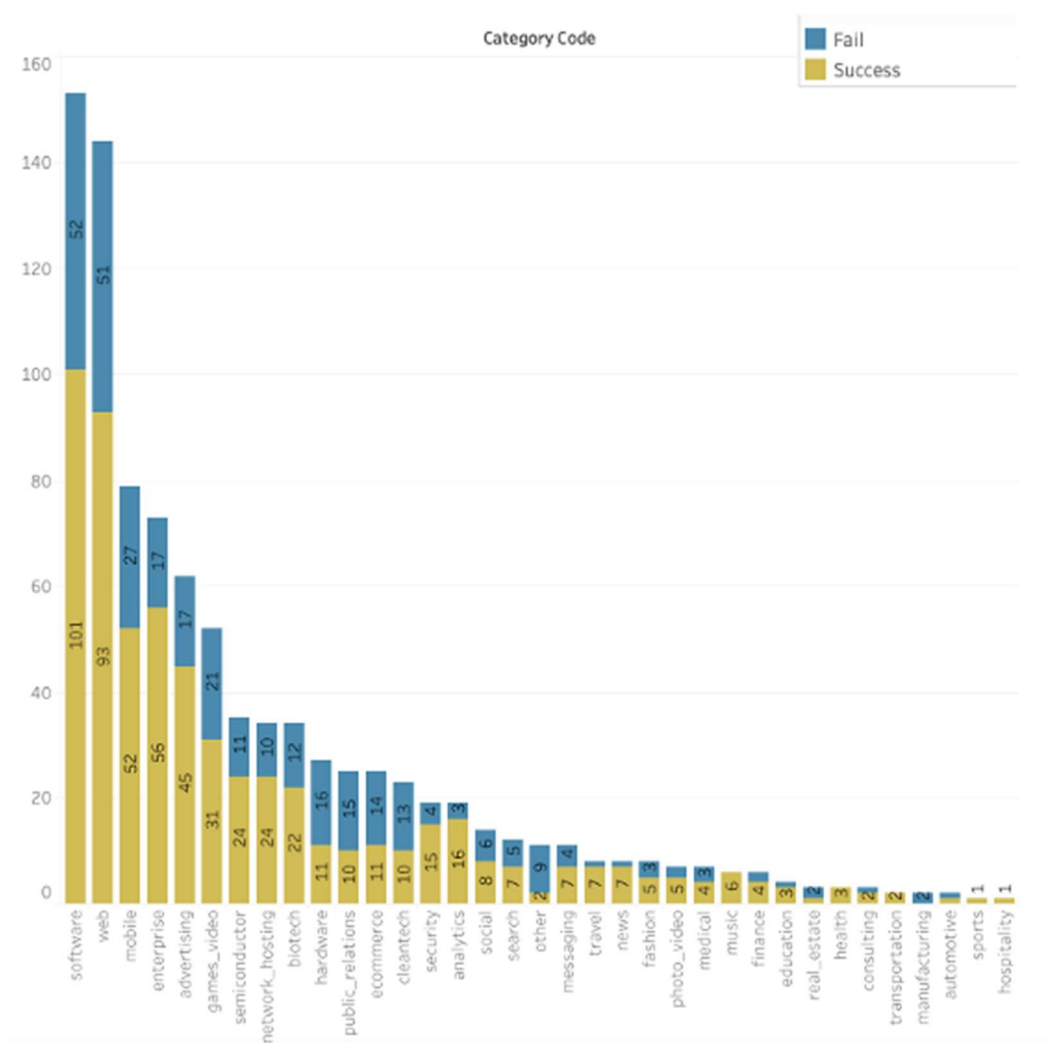


Figure 10: Startup category vs. success/failure status

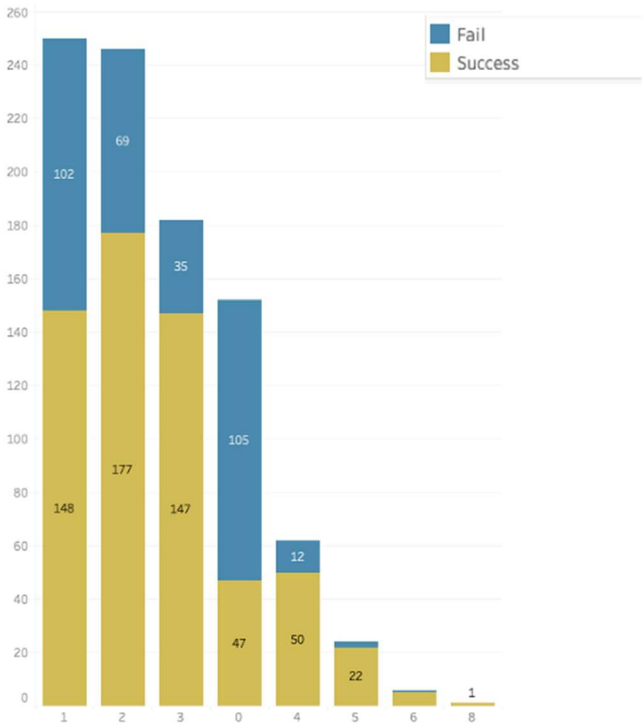


Figure 11: Milestones vs success/failure status

Table 6: Success and failure rates of startup based on funding rounds and funding type

Funding Type	Fail %	Success %
Round A (Yes)	44.3	55.7
Round A (No)	26.7	73.3
Round B (Yes)	43.3	56.7
Round B (No)	22.9	77.1
Round C (Yes)	39.7	60.3
Round C (No)	20.9	79.1
Round D (Yes)	37.5	62.5
Round D (No)	15.2	84.8
VC Funding (Yes)	33.4	66.6
VC Funding (No)	39.2	60.8
Angel Funding (Yes)	33.3	66.7
Angel Funding (No)	41.3	58.7

Table 7: Average funding rounds and total funding by investment type

Funding Type	Avg. Funding Rounds	Avg. Total Funding (USD)
No VC	1.99	\$24.1 M
VC-backed	2.98	\$28.2 M
No Angel	2.30	\$32.0 M
Angel-backed	2.34	\$6.1 M

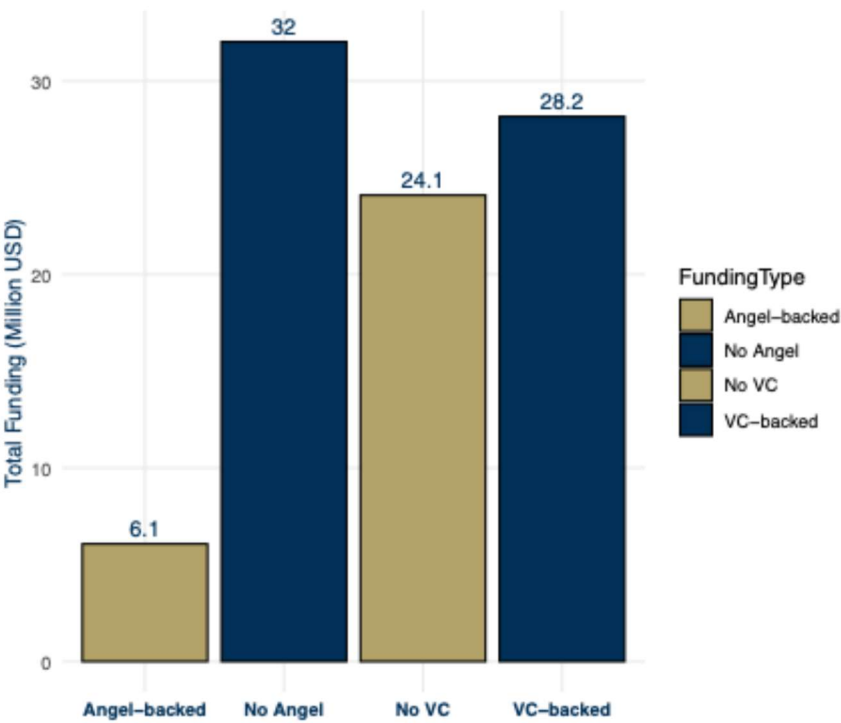


Figure 12: Average total funding by investment type

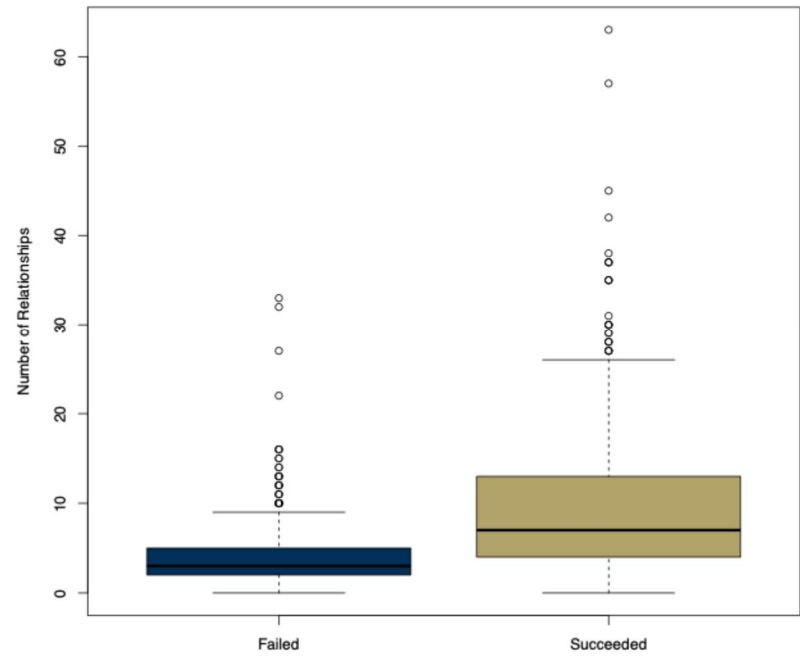


Figure 13: Relationships by startup outcomes

Table 8: Table showing all the variables and their descriptions and data type

Category	Variable (explanation)	Data Type
Category/Flag	category_code (Primary sector/category of business), is_software (Binary flag for software sector), is_web (Binary flag for web sector), is_mobile (Binary flag for mobile sector), is_enterprise (Binary flag for enterprise sector), is_advertising (Binary flag for advertising sector), is_gamesvideo (Binary flag for games/video sector), is_ecommerce (Binary flag for ecommerce sector), is_biotech (Binary flag for biotech sector), is_consulting (Binary flag for consulting sector), is_othercategory (Binary flag for other categories)	int, object
Company id	name (Company name)	object
Date/Time	founded_at (Date company was founded), closed_at (Date company closed (if applicable)), first_funding_at (Date of first funding round), last_funding_at (Date of most recent funding round)	object
Elapsed/Metrics	age_first_funding_year (Years from founding to first funding), age_last_funding_year (Years from founding to last funding), age_first_milestone_year (Years from founding to first milestone), age_last_milestone_year (Years from founding to last milestone), avg_participants (Average number of funders/participants per round)	float
Finance	funding_rounds (Number of funding rounds), funding_total_usd (Total funding amount in USD), has_VC (Binary: has venture capital funding), has_angel (Binary: has angel investor funding), has_roundA (Binary flag for round A), has_roundB (Binary flag for round B), has_roundC (Binary flag for round C), has_roundD (Binary flag for round D)	int
Geography	is_CA (Binary indicator if company is in CA), is_NY (Binary indicator if company is in NY), is_MA (Binary indicator if company is in MA), is_TX (Binary indicator if company is in TX), is_otherstate (Binary indicator for all other states), state_code (US state abbreviation), latitude (Latitude of company headquarters), longitude (Longitude of company headquarters), zip_code (Postal code), city (City of company headquarters)	object, float, int
Relations	relationships (Number of noted business connections/relationships)	int

Status/Target	milestones (Number of company milestones recorded), is_top500 (Binary is ranked top 500), status (Company outcome (e.g., acquired, closed))	int, object
---------------	---	-------------

Bibliography and Credits

1. <https://news.crunchbase.com/venture/state-of-startups-q2-h1-2025-ai-ma-charts-data/>
2. <https://www.demandsage.com/startup-statistics/>
3. Tableau Software. (2023). *Tableau Desktop* (Version 2023.x) [Computer software]. Tableau Software. <https://www.tableau.com>
4. Potanin, A., Bracci, A., Pavan, M., & Wójcik, M. (2023). Startup success prediction and VC portfolio simulation using Crunchbase data. arXiv preprint arXiv:2309.15552. <https://arxiv.org/pdf/2309.15552>
5. Sharchilev, B., Gusev, G., Serdyukov, P., & de Rijke, M. (2018). Web-based startup success prediction. Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM '18). <https://staff.fnwi.uva.nl/m.derijke/wp-content/papercite-data/pdf/sharchilev-web-based-2018.pdf>
6. Wärnberg Gerdin, J. (2022). Predicting success in early-stage start-ups using machine learning (Master's thesis, Uppsala University). DiVA Portal. <https://www.diva-portal.org/smash/get/diva2:1736961/FULLTEXT01.pdf>

Code:

```
#!/usr/bin/env python3
"""

Includes:
- Data cleaning
- Preprocessing
- Model training (RF, LR, SVM, KNN, DT, GB, MLP)
- Stacking hybrid model
- Save sensitivity and specificity of all models
"""

import warnings
warnings.filterwarnings("ignore")

import os
import pickle
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

```
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier,
    StackingClassifier
)
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, roc_curve
)
```

```
# -----
# Custom RF hyperparameters
# -----
```

```
RF_PARAMS = dict(
    n_estimators=200,
    max_depth=10,
    min_samples_split=10,
    min_samples_leaf=1,
    max_features='sqrt',
    bootstrap=False,
    random_state=42
)
```

```
# -----
# Utilities
# -----
def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
```

```

out = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, zero_division=0),
    "recall": recall_score(y_test, y_pred, zero_division=0),
    "f1": f1_score(y_test, y_pred, zero_division=0),
    "confusion_matrix": confusion_matrix(y_test, y_pred).tolist()
}

try:
    prob = model.predict_proba(X_test)[: ,1]
    out["auc"] = roc_auc_score(y_test, prob)
except:
    out["auc"] = float("nan")

return out

def save_sens_spec(model, name, X_test, y_test, save_dir="roc_metrics"):
    """
    Saves sensitivity (TPR) and specificity (TNR) for all thresholds of a model.
    """
    os.makedirs(save_dir, exist_ok=True)

    try:
        prob = model.predict_proba(X_test)[: , 1]
    except:
        print(f"Skipping {name} (no predict_proba).")
        return

    fpr, tpr, thresholds = roc_curve(y_test, prob)
    specificity = 1 - fpr # TNR = 1 - FPR

    df = pd.DataFrame({
        "threshold": thresholds,
        "sensitivity": tpr,
        "specificity": specificity
    })

    df.to_csv(f"{save_dir}/{name}_sens_spec.csv", index=False)

# -----
# Main pipeline
# -----

```

```

def main():

    df = pd.read_csv("startup111.csv")

    target = "labels"
    y = df[target]
    X = df.drop(columns=[target])

    num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    cat_cols = X.select_dtypes(include=["object", "bool"]).columns.tolist()

    pre = ColumnTransformer([
        ("num", Pipeline([
            ("imp", SimpleImputer(strategy="median")),
            ("sc", StandardScaler())
        ]), num_cols),
        ("cat", Pipeline([
            ("imp", SimpleImputer(strategy="most_frequent")),
            ("oh", OneHotEncoder(handle_unknown="ignore"))
        ]), cat_cols)
    ])

    # -----
    # Train/test split
    # -----
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y
    )

    # -----
    # Base models
    # -----
    models = {
        "RandomForest": RandomForestClassifier(**RF_PARAMS),
        "LogisticRegression": LogisticRegression(max_iter=2000),
        "GradientBoosting": GradientBoostingClassifier(),
        "SVM": SVC(probability=True),
        "KNN": KNeighborsClassifier(),
        "DecisionTree": DecisionTreeClassifier(),
        "MLP": MLPClassifier(max_iter=800)
    }

```

```

trained = {}

# -----
# Train base models
# -----
for name, model in models.items():
    print(f"Training {name}...")
    pipe = Pipeline([("pre", pre), ("clf", model)])
    pipe.fit(X_train, y_train)
    trained[name] = pipe

# -----
# Stacking Hybrid Model
# -----
print("\nTraining Hybrid Stacking Model...")

stack = StackingClassifier(
    estimators=[
        ('rf', models["RandomForest"]),
        ('gb', models["GradientBoosting"]),
        ('lr', models["LogisticRegression"]),
    ],
    final_estimator=RandomForestClassifier(**RF_PARAMS),
    passthrough=True,
    n_jobs=-1
)

hybrid = Pipeline([("pre", pre), ("clf", stack)])
hybrid.fit(X_train, y_train)
trained["Hybrid"] = hybrid

# -----
# Evaluate all models
# -----
print("\n==== MODEL PERFORMANCE =====")
results = {}

for name, model in trained.items():
    res = evaluate(model, X_test, y_test)
    results[name] = res
    print(f"\n{name}:")
    for k,v in res.items():

```

```

        print(f" {k}: {v}")

# -----
# Save sensitivity and specificity
# -----
print("\nSaving sensitivity and specificity...")
for name, model in trained.items():
    save_sens_spec(model, name, X_test, y_test)

# -----
# Save artifacts
# -----
with open("startup_artifacts_no_kmeans_custom_rf.pkl", "wb") as f:
    pickle.dump(trained, f)

print("\nArtifacts saved to startup_artifacts_no_kmeans_custom_rf.pkl")
print("Sensitivity/specificity CSVs saved to roc_metrics/")

# Run
if __name__ == "__main__":
    main()

```

Logistic Regression -Random Forest (Python Code)

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

startup=pd.read_csv('startup11.csv')
startup.head(5)
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.model_selection import StratifiedKFold,cross_val_score
startup1=startup.copy()
removed=['zip_code', 'id', 'city', 'name',
        'founded_at', 'closed_at', 'first_funding_at', 'last_funding_at','is_top500'
]
startup1=startup1.drop(columns=removed)
startup1['state_code']=startup1['state_code'].astype('category')
startup1['category_code']=startup1['category_code'].astype('category')

MODELING
y=startup1['labels']
X=startup1[[col for col in startup1.columns if col!='labels']]
X_train, X_test, y_train, y_test= train_test_split( X, y, test_size=0.3, random_state=42, stratify=y)

```

LOGISTIC REGRESSION

```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score, f1_score
scaler=StandardScaler()
num_cols=[
    'latitude',
    'longitude',
    'age_first_funding_year',
    'age_last_funding_year',
    'age_first_milestone_year',
    'age_last_milestone_year',
    'funding_total_usd',
    'avg_participants']
X_train[num_cols]=scaler.fit_transform(X_train[num_cols])
X_test[num_cols]=scaler.transform(X_test[num_cols])
cat_cols=['state_code','category_code']
X_train_encoded=pd.get_dummies(X_train,columns=cat_cols, drop_first=True)
X_test_encoded=pd.get_dummies(X_test,columns=cat_cols, drop_first=True)
from sklearn.linear_model import LogisticRegression
#add class_weighted='balance' as parameter to test for balanced model and to that for RF too.
logreg=LogisticRegression(max_iter=1000, random_state=42)
logreg.fit(X_train_encoded, y_train)
y_pred=logreg.predict(X_test_encoded)
y_pred_prob=logreg.predict_proba(X_test_encoded)[:,-1]
LRroc_auc=roc_auc_score(y_test, y_pred_prob)
LRroc_auc
from sklearn.metrics import f1_score
LRf1=f1_score(y_test, y_pred)
print(classification_report(y_test, y_pred))
Stratified
skf=StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
roc_auc_scores=cross_val_score(logreg, X_train_encoded, y_train, cv=skf, scoring='roc_auc')
f1_scores=cross_val_score(logreg, X_train_encoded, y_train, cv=skf, scoring='f1')
print("Roc_auc CV scores:", roc_auc_scores)
print("mean roc_auc:", roc_auc_scores.mean())
print("f1 cv scores:", f1_scores)
print("mean f1 cv score:", f1_scores.mean())
optimal regularization strength(C) for hyperparameter tuning
from sklearn.model_selection import GridSearchCV
param_grid={'C': [0.01,0.1,1,10,100], 'penalty':['l2'], 'solver':['lbfgs']}
grid=GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5, scoring='roc_auc')
grid.fit(X_train_encoded, y_train)
best_model=grid.best_estimator_
print(grid.best_params_)
```

```

# Predictions
y_pred_log = best_model.predict(X_test_encoded)
y_pred_prob_log = best_model.predict_proba(X_test_encoded)[:, 1]
# Metrics
roc_auc_log = roc_auc_score(y_test, y_pred_prob_log)
f1_log = f1_score(y_test, y_pred_log)
accuracy_log = accuracy_score(y_test, y_pred_log)

print("ROC-AUC:", roc_auc_log)
print("F1-score:", f1_log)
print("Accuracy:", accuracy_log)
###
from sklearn.metrics import confusion_matrix
# Confusion matrix for LR
tn, fp, fn, tp = confusion_matrix(y_test, y_pred_log).ravel()
# Sensitivity (TPR, Recall)
sensitivity_log = tp / (tp + fn)
# Specificity (TNR)
specificity_log = tn / (tn + fp)
print("Sensitivity:", sensitivity_log)
print("Specificity:", specificity_log)
FEATURE IMPORTANCE
coeffs=pd.DataFrame({'features': X_train_encoded.columns, 'coef':logreg.coef_[0]})
coeffs['abs_coef']=coeffs['coef'].abs()
coeffs.sort_values('abs_coef', ascending=False)
coeffs['Effect']=coeffs['coef'].apply(lambda x: 'Positive' if x>0 else 'Negative' )
coeffs.sort_values('abs_coef', ascending=False).head(5)
import seaborn as sns
plt.figure(figsize=(10,6))
sns.barplot(x='abs_coef', y='features', data=coeffs.head(10)) #, hue='Effect', dodge=False)
plt.title('Top 10 Logistic Regression Features by Absolute Coefficient')
plt.xlabel('Absolute Coefficient')
plt.show()

```

RANDOM FOREST

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
X_rf=X.copy()
for col in ['state_code','category_code']:
    le=LabelEncoder()
    X_rf[col]=le.fit_transform(X_rf[col])
le_state=LabelEncoder()
le_category = LabelEncoder()
X_train['state_code']=le_state.fit_transform(X_train['state_code'])
X_test['state_code']=le_state.fit_transform(X_test['state_code'])

```

```

X_train['category_code']=le_category.fit_transform(X_train['category_code'])
X_test['category_code']=le_category.fit_transform(X_test['category_code'])
rf=RandomForestClassifier( random_state=42)
rf.fit(X_train, y_train)
y_pred_rf=rf.predict(X_test)
y_pred_prob_rf=rf.predict_proba(X_test)[:,:1]
rf_roc_auc=roc_auc_score(y_test, y_pred_prob_rf)
rf_f1=f1_score(y_test, y_pred_rf)
print("roc_auc:",rf_roc_auc)
print("f1:",rf_f1)
print(classification_report(y_test, y_pred_rf))
tune parameters for RF with RandomizedSearchCV
from sklearn.model_selection import RandomizedSearchCV
param_dist={
    'n_estimators':[100,200,300,400,500],
    'max_depth':[None, 5,10,15,20],
    'min_samples_split': [2,5,10],
    'min_samples_leaf': [1,2,4,6],
    'max_features': ['sqrt','log2', None],
    'bootstrap': [True, False]
}
random_search=RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=50,
    scoring='roc_auc',
    cv=5,
    verbose=2,
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_train, y_train)
print('best hyperparameters:', random_search.best_params_)
best_rf=random_search.best_estimator_
y_pred_rf2=best_rf.predict(X_test)
y_pred_prob_rf2=best_rf.predict_proba(X_test)[:,:1]
rf_roc_auc2=roc_auc_score(y_test, y_pred_prob_rf2)
rf_f12=f1_score(y_test, y_pred_rf2)
print('roc_auc:',rf_roc_auc2)
print('f1_score:', rf_f12)
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred_rf2)
print("accuracy:", accuracy)
print(classification_report(y_test, y_pred_rf2))
# Predictions at threshold = 0.5

```

```

y_pred_class = (y_pred_prob_rf2 >= 0.5).astype(int)
# Confusion matrix
tn, fp, fn, tp = confusion_matrix(y_test, y_pred_class).ravel()
# Sensitivity (Recall, TPR)
sensitivity = tp / (tp + fn)

# Specificity (TNR)
specificity = tn / (tn + fp)
print("Sensitivity:", sensitivity)
print("Specificity:", specificity)
Feature importance for RF
importances=best_rf.feature_importances_
feature_names=X_train.columns
feat_imp_df=pd.DataFrame({
    'feature': feature_names,
    'importance': importances
})
feat_imp_df=feat_imp_df.sort_values(by='importance', ascending=False)
print(feat_imp_df.head(5))
plt.figure(figsize=(10,5))
plt.barh(feat_imp_df['feature'][:10][::-1], feat_imp_df['importance'][:10][::-1])
plt.xlabel('Importance')
plt.title('Top 10 Feature Importance-Random Forest')
plt.show()
#%%%
from sklearn.tree import plot_tree
estimator=best_rf.estimators_[3] #pick one tree
plt.figure(figsize=(25,15))
plot_tree(estimator,
    feature_names=X_train.columns,
    class_names=['Not Successful', 'Successful'],
    filled=True,
    rounded=True,
    max_depth=3)
plt.title('Random Forest: Sample Tree (depth=3)')
plt.show()

```

```

library(reshape2)
library(ggplot2)
library(tidyverse)
library(hrbrthemes)

```

```
library(viridis)
library(lars)
library(leaps)
library(pls)
library(corrplot)
library(klaR)
library(psych)
library(MASS)
#library(ggord)
library(devtools)
library(e1071)
library(dplyr)
library(caret)
library(class)
library(survival)
#install.packages("mlogit")
library(mlogit)
library(randomForest)
library(tidyverse)
library(gbm)
library(np)
library(readr)
library(lubridate)
library(rpart)
library(rpart.plot)
library(xgboost)
library(pROC)
#library(DMwR)
library(themis)
library(recipes)
library(rsample)
library(yardstick)
library(naivebayes)
set.seed(123)
### Read the data
#df <- read.table(file = "C://Users/ancha/OneDrive - Georgia Institute of
Technology/Documents/ISYE_7406_project/Mid_term_report/startup_data.csv", sep=";",
header=TRUE)
```

```
#df <- read_csv("C://Users/ancha/OneDrive - Georgia Institute of  
Technology/Documents/ISYE_7406_project/Mid_term_report/startup_data.csv")
```

```
#df <- read_csv("C://Users/ancha/OneDrive - Georgia Institute of  
Technology/Documents/ISYE_7406_project/Mid_term_report/startup_data.csv",  
show_col_types = FALSE, na = c("", "NA", "NULL"))
```

```
startups <- read_csv("C://Users/ancha/OneDrive - Georgia Institute of  
Technology/Documents/ISYE_7406_project/startup111.csv", show_col_types = FALSE, na =  
c("", "NA", "NULL"))
```

```
gc()  
head(startups)  
n = dim(startups)[1]; ### total number of observations  
print(n)  
m = dim(startups)[2]; ### total number of factors  
print(m)
```

```
gc()  
# Convert labels to factor  
startups$labels <- factor(startups$labels, levels = c("0", "1"), labels = c("Closed", "Acquired"))
```

```
# Class distribution  
print(round(prop.table(table(startups$labels)) * 100, 2))
```

```
gc()
```

```
# Train, test split 70-30  
split <- initial_split(startups, prop = 0.70, strata = labels)  
train_data <- training(split)  
test_data <- testing(split)  
gc()
```

```
head(train_data)  
head(test_data)  
gc()
```

```

# 5 fold CV on training data
ctrl <- trainControl(method = "cv", number = 5, classProbs = TRUE, summaryFunction =
twoClassSummary, savePredictions = "final", sampling = NULL)
gc()

#####
## 1. With SMOTE ##

# Preprocessing with SMOTE (only for training data)
rec_smote <- recipe(labels ~ ., data = train_data)
rec_smote <- step_rm(rec_smote, latitude, longitude)
rec_smote <- step_mutate(rec_smote, state_code = as.factor(state_code), category_code =
as.factor(category_code))

rec_smote <- step_novel(rec_smote, all_nominal_predictors())
rec_smote <- step_dummy(rec_smote, all_nominal_predictors(), one_hot = FALSE)
rec_smote <- step_zv(rec_smote, all_predictors())
rec_smote <- step_corr(rec_smote, all_predictors(), threshold = 0.95)
rec_smote <- step_center(rec_smote, all_numeric_predictors())
rec_smote <- step_scale(rec_smote, all_numeric_predictors())
rec_smote <- step_smote(rec_smote, labels, over_ratio = 1.0)
gc()
prep_smote <- prep(rec_smote)
train_smote <- bake(prep_smote, new_data = NULL)
test_smote <- bake(prep_smote, new_data = test_data)

#class distribution with SMOTE
print(round(prop.table(table(train_smote$labels))*100, 2))

# XGBoost matrices
x_train_s <- as.matrix(subset(train_smote, select = -labels))
y_train_s <- as.numeric(train_smote$labels) - 1

x_test_s <- as.matrix(subset(test_smote, select = -labels))
y_test_s <- as.numeric(test_smote$labels) - 1
gc()

```

```
# 1.1 Decision tree with SMOTE
```

```
grid_tree <- expand.grid(cp = seq(0, 0.03, by = 0.002))
```

```
dt_smote <- train(labels ~ ., data = train_smote, method = "rpart", trControl = ctrl, tuneGrid =  
grid_tree, metric = "ROC")
```

```
dt_prob_s <- predict(dt_smote, test_smote, type = "prob"), "Acquired"]
```

```
dt_class_s <- predict(dt_smote, test_smote)
```

```
dt_roc_s <- roc(y_test_s, dt_prob_s)
```

```
dt_cm_s <- confusionMatrix(dt_class_s, test_smote$labels, positive = "Acquired")
```

```
# Plot DT importance
```

```
dt_imp_s <- varImp(dt_smote)
```

```
print(dt_imp_s)
```

```
plot(dt_imp_s, top = 10, main = 'Decision Tree feature importance (SMOTE)')
```

```
gc()
```

```
# 1.2 Naive Bayes with SMOTE
```

```
nb_smote <- train(labels ~ ., data = train_smote, method = "naive_bayes", trControl =  
ctrl, tuneLength = 10, metric = "ROC")
```

```
nb_prob_s <- predict(nb_smote, test_smote, type = "prob"), "Acquired"]
```

```
nb_class_s <- predict(nb_smote, test_smote)
```

```
nb_roc_s <- roc(y_test_s, nb_prob_s)
```

```
nb_cm_s <- confusionMatrix(nb_class_s, test_smote$labels, positive = "Acquired")
```

```
gc()
```

```
# 1.3 XGBoost with SMOTE
```

```
dtrain_s <- xgb.DMatrix(data = x_train_s, label = y_train_s)
```

```
dtest_s <- xgb.DMatrix(data = x_test_s, label = y_test_s)
```

```
xgb_params <- list(objective = "binary:logistic", eval_metric = "auc", eta = 0.05, max_depth =  
6, subsample = 0.8, colsample_bytree = 0.8, min_child_weight = 1)
```

```
cv_s <- xgb.cv(params = xgb_params, data = dtrain_s, nrounds = 2000, nfold = 5,
early_stopping_rounds = 50, verbose = 0, maximize = TRUE)
```

```
best_iter_s <- cv_s$best_iteration
```

```
xgb_smote <- xgb.train(params = xgb_params, data = dtrain_s, nrounds = best_iter_s,
verbose = 0)
```

```
xgb_prob_s <- predict(xgb_smote, dtest_s)
```

```
xgb_class_s <- factor(ifelse(xgb_prob_s > 0.5, "Acquired", "Closed"), levels = c("Closed",
"Acquired"))
```

```
xgb_roc_s <- roc(y_test_s, xgb_prob_s)
```

```
xgb_cm_s <- confusionMatrix(xgb_class_s, test_smote$labels, positive = "Acquired")
```

```
gc()
```

```
# 1.4 Simple average hybrid with SMOTE
```

```
hybrid_prob_s <- (dt_prob_s + nb_prob_s + xgb_prob_s) / 3
```

```
hybrid_class_s <- factor(ifelse(hybrid_prob_s > 0.5, "Acquired", "Closed"), levels =
c("Closed", "Acquired"))
```

```
hybrid_roc_s <- roc(y_test_s, hybrid_prob_s)
```

```
hybrid_cm_s <- confusionMatrix(hybrid_class_s, test_smote$labels, positive = "Acquired")
```

```
# Plot XGBoost importance
```

```
xgb_imp_s <- xgb.importance(colnames(x_train_s), model = xgb_smote)
```

```
xgb.plot.importance(xgb_imp_s, top_n = 10, main = 'XGBoost feature importance (SMOTE)')
```

```
# Plots for SMOTE
```

```
ggroc(list(DT = dt_roc_s, NB = nb_roc_s, XGB = xgb_roc_s, Hybrid = hybrid_roc_s), legacy.axes
= TRUE) + ggtitle("ROC curves (with SMOTE)") + theme_minimal()
```

```
#####
```

```
## 2. Without SMOTE ##
```

```

rec_nosmote <- recipe(labels ~ ., data = train_data)
rec_nosmote <- step_rm(rec_nosmote, latitude, longitude)
rec_nosmote <- step_mutate(rec_nosmote, state_code = as.factor(state_code),
category_code = as.factor(category_code))
rec_nosmote <- step_novel(rec_nosmote, all_nominal_predictors())
rec_nosmote <- step_dummy(rec_nosmote, all_nominal_predictors(), one_hot = FALSE)
rec_nosmote <- step_zv(rec_nosmote, all_predictors())
rec_nosmote <- step_corr(rec_nosmote, all_predictors(), threshold = 0.95)
rec_nosmote <- step_center(rec_nosmote, all_numeric_predictors())
rec_nosmote <- step_scale(rec_nosmote, all_numeric_predictors())

```

```

prep_nosmote <- prep(rec_nosmote)
train_nosmote <- bake(prep_nosmote, new_data = NULL)
test_nosmote <- bake(prep_nosmote, new_data = test_data)

```

```
gc()
```

```

#class distribution without SMOTE
print(round(prop.table(table(train_nosmote$labels))*100, 2))

```

```

x_train_n <- as.matrix(subset(train_nosmote, select = -labels))
y_train_n <- as.numeric(train_nosmote$labels) - 1
x_test_n <- as.matrix(subset(test_nosmote, select = -labels))
y_test_n <- as.numeric(test_nosmote$labels) - 1

```

```

gc()
# 2.1 Decision Tree without SMOTE
dt_no <- train(labels ~ ., data = train_nosmote, method = "rpart", trControl = ctrl, tuneGrid =
grid_tree, metric = "ROC")

```

```

dt_prob_n <- predict(dt_no, test_nosmote, type = "prob")["Acquired"]
dt_class_n <- predict(dt_no, test_nosmote)
dt_roc_n <- roc(y_test_n, dt_prob_n)
dt_cm_n <- confusionMatrix(dt_class_n, test_nosmote$labels, positive = "Acquired")

```

```

dt_imp_n <- varImp(dt_no)
print(dt_imp_n)
plot(dt_imp_n, top = 10, main = 'Decision Tree feature importance (no SMOTE)')

gc()
# 2.2 Naive Bayes without SMOTE
nb_no <- train(labels ~ ., data = train_nosmote, method = "naive_bayes", trControl = ctrl,
tuneLength = 10, metric = "ROC")

nb_prob_n <- predict(nb_no, test_nosmote, type = "prob"), "Acquired"]
nb_class_n <- predict(nb_no, test_nosmote)
nb_roc_n <- roc(y_test_n, nb_prob_n)
nb_cm_n <- confusionMatrix(nb_class_n, test_nosmote$labels, positive = "Acquired")

gc()
# 2.3 Xgboost without SMOTE
dtrain_n <- xgb.DMatrix(data = x_train_n, label = y_train_n)
dtest_n <- xgb.DMatrix(data = x_test_n, label = y_test_n)

cv_n <- xgb.cv(params = xgb_params, data = dtrain_n, nrounds = 2000, nfold = 5,
early_stopping_rounds = 50, verbose = 0, maximize = TRUE)

best_iter_n <- cv_n$best_iteration
xgb_no <- xgb.train(params = xgb_params, data = dtrain_n, nrounds = best_iter_n, verbose =
0)

xgb_prob_n <- predict(xgb_no, dtest_n)
xgb_class_n <- factor(ifelse(xgb_prob_n > 0.5, "Acquired", "Closed"), levels = c("Closed",
"Acquired"))
xgb_roc_n <- roc(y_test_n, xgb_prob_n)
xgb_cm_n <- confusionMatrix(xgb_class_n, test_nosmote$labels, positive = "Acquired")

xgb_imp_n <- xgb.importance(colnames(x_train_n), model = xgb_no)
xgb.plot.importance(xgb_imp_n, top_n = 10, main = 'XGBoost feature importance (no
SMOTE)')

# 2.4 Average hybrid model without SMOTE

```

```

hybrid_prob_n <- (dt_prob_n + nb_prob_n + xgb_prob_n) / 3
hybrid_class_n <- factor(ifelse(hybrid_prob_n > 0.5, "Acquired", "Closed"), levels =
c("Closed", "Acquired"))
hybrid_roc_n <- roc(y_test_n, hybrid_prob_n)
hybrid_cm_n <- confusionMatrix(hybrid_class_n, test_nosmote$labels, positive =
"Acquired")
# Plots for no SMOTE
ggroc(list(DT = dt_roc_n, NB = nb_roc_n, XGB = xgb_roc_n, Hybrid = hybrid_roc_n),
legacy.axes = TRUE) + ggtitle("ROC curves (without SMOTE)") + theme_minimal()

gc()
## Final comparison of SMOTE vs no SMOTE
#results <- data.frame(Scenario = rep(c("With SMOTE", "Without SMOTE"), each = 4), Model
= rep(c("Decision Tree", "Naive Bayes", "XGBoost", "Hybrid Average"), 2), AUC =
round(c(auc(dt_roc_s), auc(nb_roc_s), auc(xgb_roc_s), auc(hybrid_roc_s), auc(dt_roc_n),
auc(nb_roc_n), auc(xgb_roc_n), auc(hybrid_roc_n)), 4), F1 =
round(c(dt_cm_s$byClass["F1"], nb_cm_s$byClass["F1"], xgb_cm_s$byClass["F1"],
hybrid_cm_s$byClass["F1"], dt_cm_n$byClass["F1"], nb_cm_n$byClass["F1"],
xgb_cm_n$byClass["F1"], hybrid_cm_n$byClass["F1"]), 4), Sensitivity =
round(c(dt_cm_s$byClass["Sensitivity"], nb_cm_s$byClass["Sensitivity"],
xgb_cm_s$byClass["Sensitivity"], hybrid_cm_s$byClass["Sensitivity"],
dt_cm_n$byClass["Sensitivity"], nb_cm_n$byClass["Sensitivity"],
xgb_cm_n$byClass["Sensitivity"], hybrid_cm_n$byClass["Sensitivity"]), 4), Specificity =
round(c(dt_cm_s$byClass["Specificity"], nb_cm_s$byClass["Specificity"],
xgb_cm_s$byClass["Specificity"], hybrid_cm_s$byClass["Specificity"],
dt_cm_n$byClass["Specificity"], nb_cm_n$byClass["Specificity"],
xgb_cm_n$byClass["Specificity"], hybrid_cm_n$byClass["Specificity"]), 4))

#results <- results[order(results$Scenario, -results$AUC), ]
#row.names(results) <- NULL

#print(results, row.names = FALSE)
#gc()
# Metrics
calc_metrics <- function(cm) {
  TP <- cm$table[2,2]; TN <- cm$table[1,1]; FP <- cm$table[2,1]; FN <- cm$table[1,2]
  Accuracy <- (TP + TN) / sum(cm$table)
  Precision <- ifelse(TP + FP == 0, 0, TP / (TP + FP))
  Recall <- ifelse(TP + FN == 0, 0, TP / (TP + FN))

```

```

F1      <- ifelse(Precision + Recall == 0, 0, 2 * Precision * Recall / (Precision + Recall))
Specificity <- ifelse(TN + FP == 0, 0, TN / (TN + FP))
c(Accuracy = Accuracy, Precision = Precision, Recall = Recall, F1 = F1, Specificity =
Specificity)
}

gc()

# Results table
results <- data.frame(
  Scenario = rep(c("With SMOTE", "Without SMOTE"), each = 4),
  Model = rep(c("Decision Tree", "Naive Bayes", "XGBoost", "Hybrid"), 2),
  AUC = round(c(auc(dt_roc_s), auc(nb_roc_s), auc(xgb_roc_s), auc(hybrid_roc_s),
                auc(dt_roc_n), auc(nb_roc_n), auc(xgb_roc_n), auc(hybrid_roc_n)), 4)
)

# Add metrics to list
metrics_list <- list(
  calc_metrics(dt_cm_s),      calc_metrics(nb_cm_s),      calc_metrics(xgb_cm_s),
  calc_metrics(hybrid_cm_s),
  calc_metrics(dt_cm_n),      calc_metrics(nb_cm_n),      calc_metrics(xgb_cm_n),
  calc_metrics(hybrid_cm_n)
)

gc()

results$Accuracy <- round(sapply(metrics_list, "[", "Accuracy"), 4)
results$Precision <- round(sapply(metrics_list, "[", "Precision"), 4)
results$Recall <- round(sapply(metrics_list, "[", "Recall"), 4)
results$F1 <- round(sapply(metrics_list, "[", "F1"), 4)
results$Specificity <- round(sapply(metrics_list, "[", "Specificity"), 4)

results <- results[order(results$Scenario, -results$AUC), ]
row.names(results) <- NULL
gc()

print(results)
# Export ROC curves

# Folder

```

```
output_folder <- "C:/Users/ancha/OneDrive - Georgia Institute of  
Technology/Documents/ISYE_7406_project/Final_Report/roc_csv_ancha"
```

```
# SIMPLE FUNCTION — NO PIPES, NO DPLYR NEEDED
```

```
export_roc_to_csv <- function(roc_obj, model_name, scenario) {
```

```
  # Step 1: Get all points from the ROC curve
```

```
  roc_matrix <- coords(roc_obj, "all",  
    ret = c("threshold", "sensitivity", "specificity",  
    "ppv", "npv", "tp", "fp", "tn", "fn"),  
    transpose = FALSE)
```

```
  # Step 2: Convert matrix to data frame
```

```
  roc_data <- as.data.frame(roc_matrix)
```

```
  # Step 3: Add False Positive Rate (FPR)
```

```
  roc_data$fpr <- 1 - roc_data$specificity
```

```
  # Step 4: Keep only the columns we want (in nice order)
```

```
  roc_data <- roc_data[c("threshold", "sensitivity", "specificity", "fpr",  
    "tp", "fp", "tn", "fn", "ppv", "npv")]
```

```
  # Step 5: Sort by threshold (from 0 to 1)
```

```
  roc_data <- roc_data[order(roc_data$threshold), ]
```

```
  # Step 6: Create filename and save
```

```
  filename <- file.path(output_folder,  
    paste0("ROC_", scenario, "_", model_name, ".csv"))
```

```
  write.csv(roc_data, filename, row.names = FALSE)
```

```
  cat("Saved:", filename, "\n")
```

```
}
```

```
gc()
```

```
# RUN ALL 8 MODELS
```

```
export_roc_to_csv(dt_roc_s, "DecisionTree", "With_SMOTE")
```

```
export_roc_to_csv(nb_roc_s, "NaiveBayes", "With_SMOTE")
```

```
export_roc_to_csv(xgb_roc_s, "XGBoost", "With_SMOTE")
export_roc_to_csv(hybrid_roc_s, "Hybrid_Ensemble", "With_SMOTE")
```

```
export_roc_to_csv(dt_roc_n, "DecisionTree", "Without_SMOTE")
export_roc_to_csv(nb_roc_n, "NaiveBayes", "Without_SMOTE")
export_roc_to_csv(xgb_roc_n, "XGBoost", "Without_SMOTE")
export_roc_to_csv(hybrid_roc_n, "Hybrid_Ensemble", "Without_SMOTE")
```

```
cat("\nAll 8 ROC curve CSV files successfully saved!\n")
cat("Location:", output_folder, "\n")
```

```
gc()
```